



LAPORAN PRAKTIKUM

TUGAS 1: RESTFUL API MURNI DENGAN EXPRESS.JS

Mata Kuliah Pengembangan Aplikasi Web II

Nama : Reva Lina
NIM : 2428240105
Kelas : SI5B
Nomor Topik : 30
Nama Topik : Sarana Olahraga: Booking Lapangan
Dosen Pengampu : Nur Rachmat, M.Kom.

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
2026**

I. Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa mampu:

1. Memahami konsep dasar RESTful API serta proses pembuatan API menggunakan Node.js dan Express.js, mulai dari melakukan inisialisasi project, membuat struktur aplikasi, hingga menjalankan aplikasi secara lokal dan melakukan deployment ke Vercel.
2. Mampu menerapkan routing RESTful pada resource /field-bookings untuk sistem booking lapangan olahraga dengan menggunakan method GET, POST, PUT, dan DELETE, serta menerapkan parameter ID dan query string sesuai dengan kebutuhan setiap endpoint.
3. Mampu menerapkan pengolahan dan validasi data pada API, seperti memastikan field yang wajib diisi tidak kosong, membuat ID secara otomatis, melakukan penambahan, perubahan, pencarian, dan penghapusan data booking, serta memberikan penanganan yang sesuai ketika terjadi kesalahan atau data tidak ditemukan.
4. Mampu menghasilkan response API dalam format JSON dengan struktur dan HTTP status code yang sesuai, kemudian melakukan pengujian terhadap setiap endpoint menggunakan tools pengujian API untuk memastikan seluruh fungsi berjalan dengan baik, baik pada server lokal maupun hasil deployment di Vercel.

II. Dasar Teori

A. REST dan Resource

REST (Representational State Transfer) merupakan pendekatan dalam perancangan layanan web yang menggunakan protokol HTTP untuk melakukan komunikasi antara client dan server. Dalam REST, data atau objek yang dikelola oleh sistem disebut sebagai resource dan dapat diakses melalui URL atau endpoint tertentu.

Pada praktikum ini, resource yang digunakan adalah field-bookings yang digunakan untuk mengelola data booking lapangan olahraga. Data booking terdiri dari nama pemesan, nama lapangan, tanggal, jam mulai, dan durasi pemakaian. API menggunakan 3 data awal sebagai data yang digunakan untuk pengujian sebelum dilakukan proses tambah, ubah, dan hapus data.

B. Method HTTP

Method HTTP digunakan untuk menentukan operasi yang dilakukan terhadap suatu resource.

Method yang digunakan pada praktikum ini yaitu:

- GET, digunakan untuk mengambil dan menampilkan data booking.
- POST, digunakan untuk menambahkan data booking baru.
- PUT, digunakan untuk memperbarui data booking berdasarkan ID.
- DELETE, digunakan untuk menghapus data booking berdasarkan ID.

C. HTTP Status Code

HTTP status code merupakan kode yang menunjukkan hasil dari request yang dikirim oleh client ke server. Status code yang digunakan dalam API ini antara lain:

- 200 OK, menunjukkan request berhasil diproses.
- 201 Created, menunjukkan data berhasil ditambahkan.
- 400 Bad Request, menunjukkan data yang dikirim tidak valid atau terdapat field wajib yang kosong.
- 404 Not Found, menunjukkan data atau endpoint yang diminta tidak ditemukan.

Penggunaan status code yang sesuai membantu client mengetahui apakah request berhasil atau mengalami kesalahan.

III. Alat dan Bahan

No	Nama Alat / Bahan	Versi / Keterangan
1	Node.js	Digunakan sebagai runtime untuk menjalankan aplikasi JavaScript pada sisi server.
2	npm (Node Package Manager)	Digunakan untuk mengelola package dan dependency yang diperlukan dalam project Node.js.
3	Express.js	Digunakan sebagai framework untuk Node.js dalam membuat server, routing, middleware, dan RESTful API.
4	Visual Studio Code	Digunakan sebagai code editor untuk membuat dan mengedit file pada project RESTful API.

No	Nama Alat / Bahan	Versi / Keterangan
5	Nodemon	Digunakan sebagai package tambahan untuk membantu menjalankan server dan melakukan restart secara otomatis ketika terdapat perubahan pada kode.
6	Postman	Digunakan untuk menguji setiap endpoint RESTful API, seperti GET, POST, PUT, dan DELETE, serta melihat response dan HTTP status code.
7	Git dan GitHub	Digunakan untuk menyimpan source code, mengelola perubahan project, dan melakukan commit secara bertahap.
8	Vercel	Digunakan untuk melakukan deployment RESTful API agar dapat diakses secara online.
9	Web Browser	Digunakan untuk mengakses endpoint API yang telah di-deploy pada Vercel dan memastikan API dapat berjalan dengan baik.

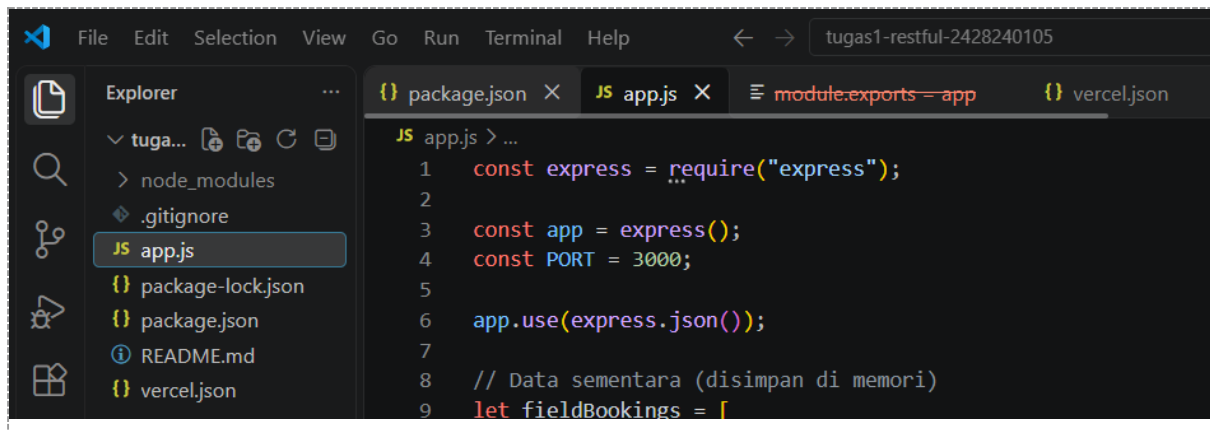
IV. Langkah Praktikum

1. Inisialisasi Project

Tahap pertama dalam praktikum adalah menyiapkan project Node.js yang akan digunakan untuk membangun RESTful API. Project dibuat menggunakan Node.js dan Express.js. Pada tahap ini digunakan nodemon sebagai dependency untuk membantu proses pengembangan dan

menjalankan server secara otomatis ketika terdapat perubahan kode. untuk membantu proses pengembangan dan menjalankan server secara otomatis ketika terdapat perubahan pada kode.

Project memiliki file utama app.js yang digunakan untuk mengatur konfigurasi server, middleware, data awal, serta endpoint RESTful API. Dependency yang digunakan dalam project tercatat pada file package.json.



Gambar 1. Struktur project dan konfigurasi awal RESTful API pada Visual Studio Code

Kode tersebut digunakan untuk mengimpor Express.js, membuat aplikasi Express, menentukan port yang digunakan oleh server, serta mengaktifkan middleware `express.json()` agar aplikasi dapat menerima request dalam format JSON.

2. Data Awal

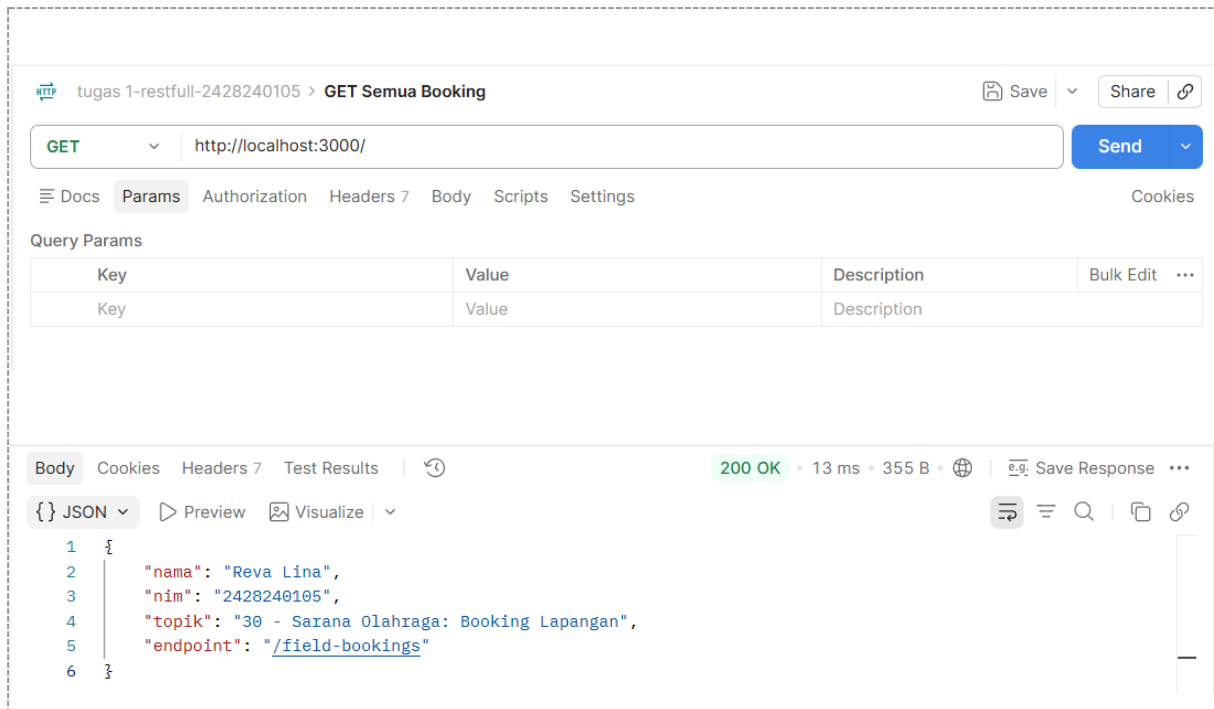
Setelah project berhasil disiapkan, tahap selanjutnya adalah membuat data awal untuk resource field-bookings. Data booking disimpan dalam bentuk array di dalam file app.js karena RESTful API yang dibuat tidak menggunakan database.

Pada tahap ini digunakan 3 data awal sebagai data yang akan digunakan untuk menguji berbagai endpoint yang telah dibuat. Setiap data booking memiliki informasi berupa ID, nama pemesan, lapangan, tanggal, jam mulai, dan durasi pemakaian.

```
9   let fieldBookings = [  
10     {  
11       id: 1,  
12       namaPemesan: "Tim Garuda SI",  
13       lapangan: "Futsal A",  
14       tanggal: "2026-10-10",  
15       jamMulai: "19:00",  
16       durasiJam: 2  
17     },  
18     {  
19       id: 2,  
20       namaPemesan: "Tim Elang",  
21       lapangan: "Futsal B",  
22       tanggal: "2026-10-11",  
23       jamMulai: "16:00",  
24       durasiJam: 2  
25     },  
26     {  
27       id: 3,  
28       namaPemesan: "Andi Saputra",  
29       lapangan: "Badminton A",  
30       tanggal: "2026-10-12",  
31       jamMulai: "18:30",  
32       durasiJam: 1  
33     }  
34   ];
```

Gambar 2. Kode data awal dan route GET / pada app.js

Data tersebut digunakan sebagai data awal sebelum dilakukan proses penambahan, perubahan, maupun penghapusan data melalui endpoint RESTful API. Variabel nextId digunakan untuk menentukan ID berikutnya secara otomatis ketika data baru ditambahkan.



Gambar 3. Hasil pengujian GET / menggunakan Postman

3. Membuat Route GET

Tahap selanjutnya adalah membuat route GET untuk mengambil data booking. Route GET digunakan untuk menampilkan seluruh data, mengambil satu data berdasarkan ID, dan melakukan filter berdasarkan nama lapangan.

Endpoint yang dibuat adalah:

1. GET `/field-bookings` untuk mengambil seluruh data booking.
2. GET `/field-bookings/:id` untuk mengambil satu data berdasarkan ID.
3. GET `/field-bookings?lapangan=nilai` untuk melakukan filter berdasarkan nama lapangan.

```
// GET /field-bookings -> menampilkan seluruh data
// Filter: /field-bookings?lapangan=Futsal A
app.get("/field-bookings", (req, res) => {
  const { lapangan } = req.query;

  if (lapangan) {
    const hasil = fieldBookings.filter(
      (b) => b.lapangan === lapangan
    );

    return res.status(200).json(hasil);
  }

  res.status(200).json(fieldBookings);
});
```

Gambar 4. Implementasi route GET untuk menampilkan seluruh data booking, melakukan filter berdasarkan lapangan, dan menampilkan data berdasarkan ID.

```
// GET /field-bookings/:id -> menampilkan satu data berdasarkan id
app.get("/field-bookings/:id", (req, res) => {
  const id = parseInt(req.params.id);

  const data = fieldBookings.find((b) => b.id === id);

  if (!data) {
    return res.status(404).json({
      status: "error",
      message: "Data tidak ditemukan",
      data: null
    });
  }

  res.status(200).json(data);
});
```

Gambar 5. Implementasi Route GET berdasarkan ID

Gambar 4 menunjukkan kode route GET /field-bookings yang digunakan untuk menampilkan seluruh data booking. Route ini juga menyediakan fitur filter berdasarkan nama lapangan menggunakan query ?lapangan=Futsal A. Jika tidak menggunakan filter, seluruh data booking akan ditampilkan dengan status 200.

Gambar 5 menunjukkan kode route GET /field-bookings/:id yang digunakan untuk menampilkan satu data booking berdasarkan ID. Sistem mencari data sesuai ID yang diberikan pada URL. Jika data ditemukan, response yang diberikan memiliki status 200, sedangkan jika data tidak ditemukan, sistem memberikan response 404 dengan format JSON.

4. Implementasi Route POST dan Validasi

Route POST digunakan untuk menambahkan data booking baru ke dalam sistem. Data yang dikirim melalui request body harus memiliki field namaPemesan, lapangan, tanggal, jamMulai, dan durasiJam.

Sebelum data ditambahkan, sistem melakukan validasi untuk memastikan seluruh field wajib sudah diisi. Jika terdapat field yang kosong, sistem memberikan response dengan status 400. Jika semua data lengkap, sistem membuat ID secara otomatis, kemudian menyimpan data booking ke dalam array dan memberikan response dengan status 201.

```
79 // POST /field-bookings -> menambahkan booking baru
80 app.post("/field-bookings", (req, res) => {
81     const { namaPemesan, lapangan, tanggal, jamMulai, durasiJam } = req.body;
82
83     if (
84         !namaPemesan ||
85         !lapangan ||
86         !tanggal ||
87         !jamMulai ||
88         durasiJam === undefined ||
89         typeof durasiJam !== "number"
90     ) { ...
91     }
92
93     const baru = {
94         id: fieldBookings.length > 0
95           ? Math.max(...fieldBookings.map((b) => b.id)) + 1
96         : 1,
97         namaPemesan,
98         lapangan,
99         tanggal,
100        jamMulai,
101        durasiJam
102    };
103
104    fieldBookings.push(baru);
105
106    res.status(201).json({
107        status: "success",
108        message: "Booking berhasil ditambahkan",
109        data: baru
110    });
111 });
```

Gambar 6. Implementasi Route POST dan Validasi Data Booking

Gambar 6 menunjukkan proses penambahan booking baru menggunakan method POST. Sistem memeriksa kelengkapan data seperti nama pemesan, lapangan, tanggal, jam mulai, dan durasi. Jika data tidak lengkap atau durasi bukan berupa angka, sistem memberikan status **400**. Jika data valid, sistem membuat ID secara otomatis, menyimpan booking baru, dan memberikan status **201**.

5. Route PUT dan DELETE

Route put digunakan untuk memperbarui data booking yang sudah ada berdasarkan ID. Pada proses ini, data yang dikirim harus lengkap karena put digunakan untuk mengganti data secara keseluruhan. Jika IID yang diberikan tidak ditemukan, sistem akan memberikan response dengan status 404.

Sedangkan route DELETE digunakan untuk menghapus data booking berdasarkan ID. Sistem akan mencari data terlebih dahulu, kemudian menghapusnya jika data ditemukan. Jika ID tidak ditemukan, sistem akan memberikan response dengan status 404.

```
118 // PUT /field-bookings/:id -> mengubah data booking
119 app.put("/field-bookings/:id", (req, res) => {
120   const id = parseInt(req.params.id);
121   const index = fieldBookings.findIndex((b) => b.id === id);
122
123   if (index === -1) {
124     return res.status(404).json({
125       status: "error",
126       message: "Data tidak ditemukan",
127       data: null
128     });
129   }
130   const { namaPemesan, lapangan, tanggal, jamMulai, durasiJam } = req.body;
131   if (
132     !namaPemesan ||
133     !lapangan ||
134     !tanggal ||
135     !jamMulai ||
136     durasiJam === undefined ||
137     typeof durasiJam !== "number"
138   ) {
139     return res.status(400).json({
140       status: "error",
141       message: "Semua field wajib diisi",
142       data: null
143     });
144   }
```

```
145   fieldBookings[index] = {
146     id: id,
147     namaPemesan,
148     lapangan,
149     tanggal,
150     jamMulai,
151     durasiJam
152   };
153   res.status(200).json({
154     status: "success",
155     message: "Booking berhasil diperbarui",
156     data: fieldBookings[index]
157   });
158 });
```

```
// DELETE /field-bookings/:id -> menghapus data booking
app.delete("/field-bookings/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const index = fieldBookings.findIndex(b => b.id === id);

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: "Data tidak ditemukan",
      data: null
    });
  }

  fieldBookings.splice(index, 1);

  res.status(200).json({
    status: "success",
    message: "Booking berhasil dihapus",
    data: null
  });
});

app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null
  });
});

if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`Server berjalan di http://localhost:${PORT}`);
  });
}
```

Gambar 7. Kode route PUT dan DELETE pada app.js

6. Middleware 404

Middleware 404 digunakan untuk menangani request yang menuju endpoint yang tidak tersedia. Jika pengguna mengakses URL atau endpoint yang tidak terdaftar pada aplikasi, sistem akan memberikan response dengan status 404 dalam format JSON. Hal ini memastikan setiap request yang tidak sesuai tetap mendapatkan response yang terstruktur.

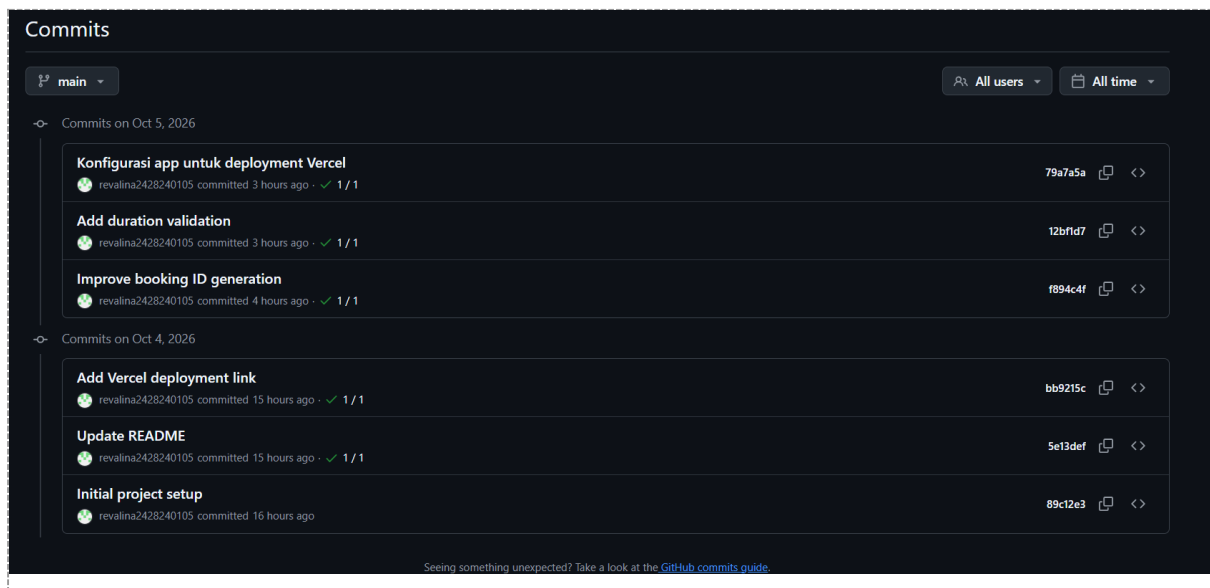
```
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null
  });
});
```

Gambar 8. Implementasi Middleware 404 untuk Endpoint Tidak Ditemukan

7. Push Project ke GitHub

Setelah project RESTful API selesai dibuat dan diuji secara lokal, project kemudian diunggah ke GitHub. GitHub digunakan untuk menyimpan source code project sekaligus mencatat perubahan yang dilakukan selama proses pengembangan.

Repository dibuat dengan nama tugas1-restful-2428240105 dan bersifat public sesuai dengan ketentuan tugas. Project diunggah menggunakan Git melalui beberapa commit yang dilakukan secara bertahap. Pada tahap ini, project telah memiliki minimal enam commit yang menunjukkan proses perubahan dan pengembangan project.

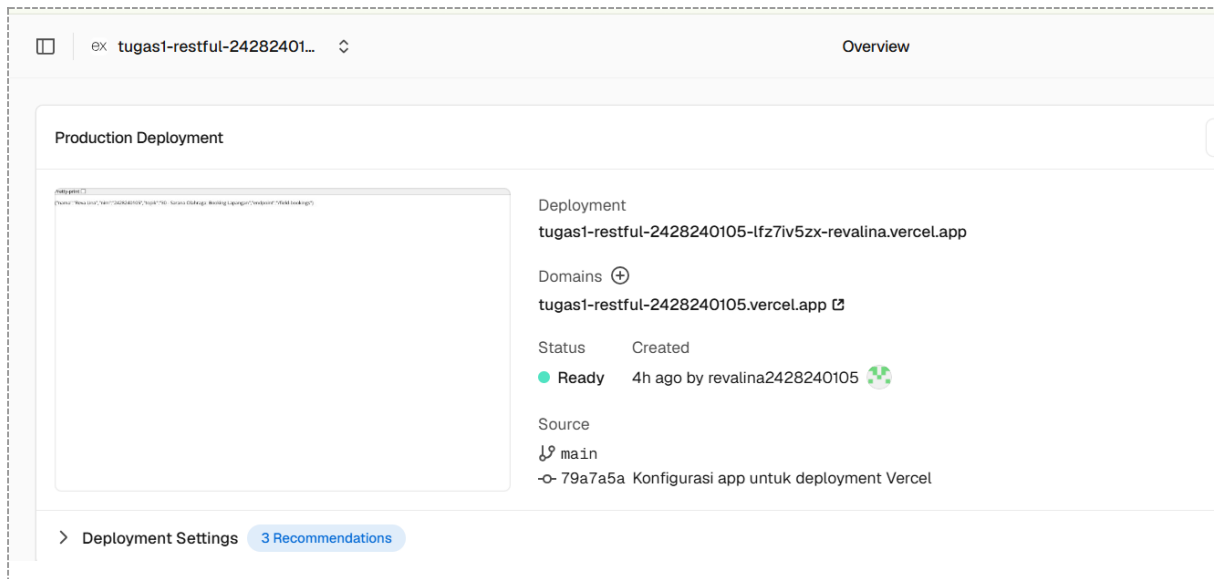


Gambar 9. Repository Project dan Riwayat Commit pada GitHub

8. Deploy ke Vercel

Setelah project berhasil diunggah ke GitHub, tahap selanjutnya adalah melakukan deployment menggunakan Vercel. Deployment dilakukan agar RESTful API dapat diakses secara online dan tidak hanya berjalan pada komputer lokal.

Project GitHub kemudian dihubungkan dengan Vercel dan dilakukan proses deployment. Setelah proses deployment berhasil, API dapat diakses melalui URL Vercel yang telah disediakan. Endpoint utama yang digunakan untuk pengujian adalah:



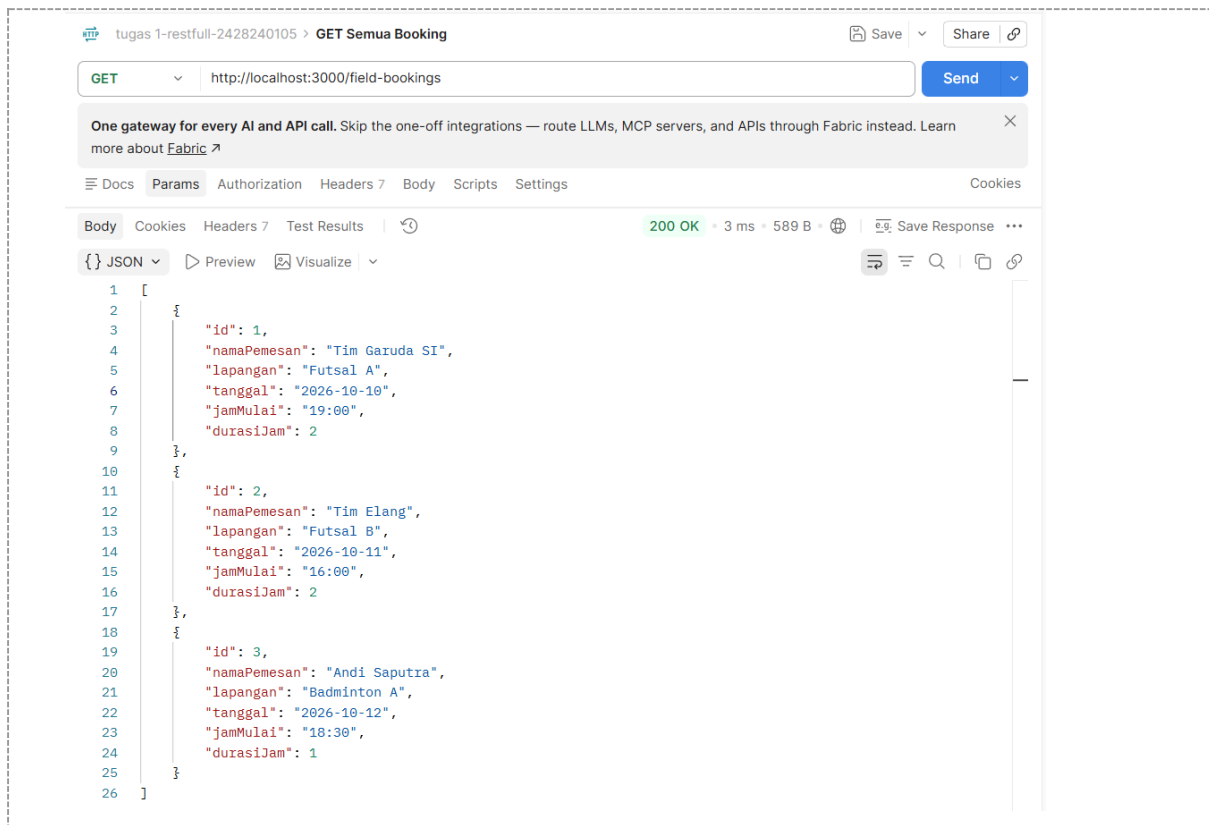
Gambar 10. Hasil Deployment Project pada Vercel

V. Hasil Pengujian

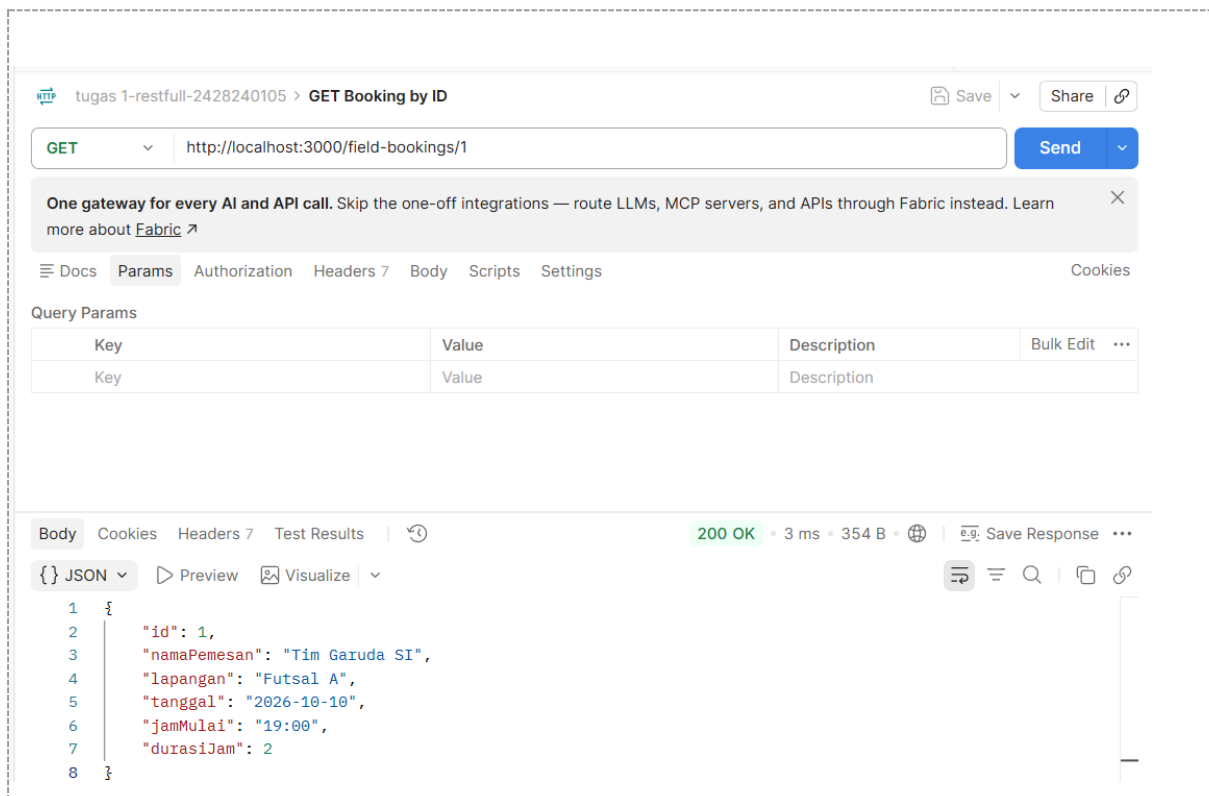
Pengujian dilakukan dengan Postman pada URL Vercel: [https://tugas1-restful-2428240105.vercel.app/]. Tabel berikut memuat 10 skenario pengujian beserta satu skenario tambahan untuk middleware catch-all.

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/field-bookings	-	200	200	Menampilkan seluruh data booking
2	GET	/field-bookings/1	-	200	200	Menampilkan data booking ID 1
3	GET	/field-bookings/99	-	404	404	ID tidak ditemukan
4	GET	/field-bookings?lapangan=Futsal A	Filter Lapangan	200	200	Menampilkan booking pada lapangan Futsal A

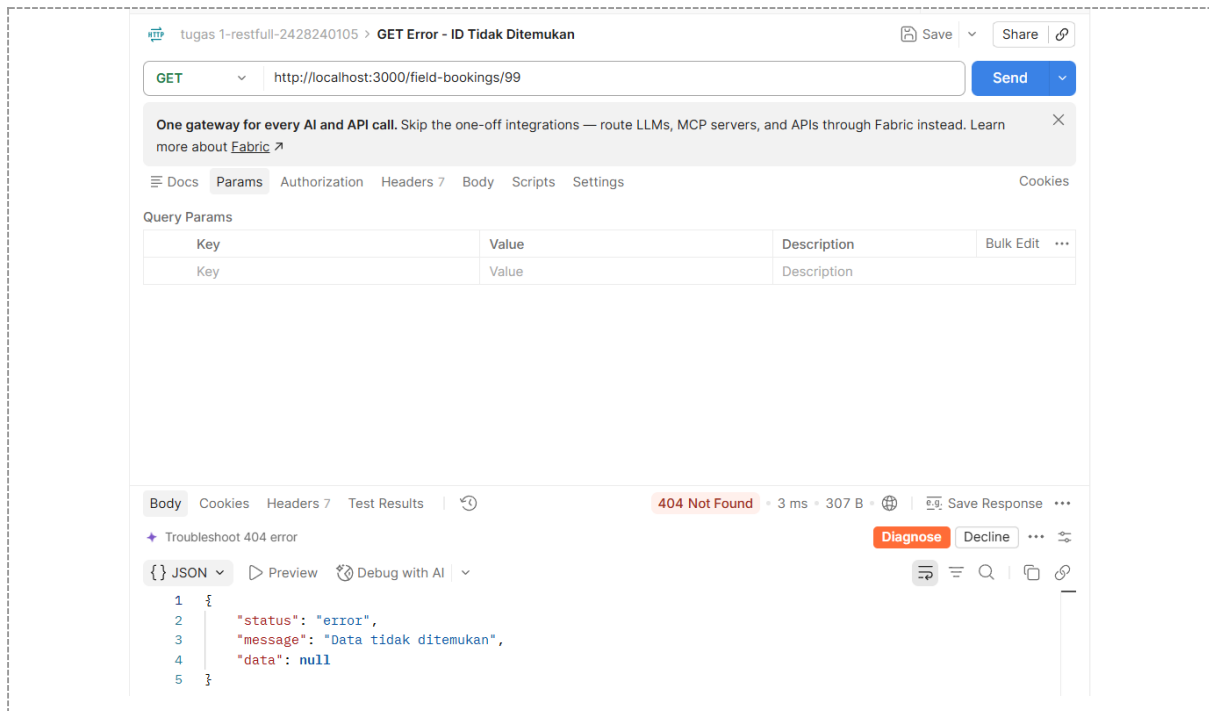
No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
5	POST	/field-bookings	Body Lengkap	201	201	Menambahkan data booking
6	POST	/field-bookings	Field wajib kosong	400	400	Menolak data yang tidak lengkap
7	PUT	/field-bookings/1	Body lengkap	200	200	Memperbarui data ID 1
8	PUT	/field-bookings/99	Body lengkap	404	404	ID tidak ditemukan
9	DELETE	/field-bookings/1	-	200	200	Menghapus data ID 1
10	DELETE	/field-bookings/99	-	404	404	ID tidak ditemukan
11	GET	/abc (catch-all)	—	404	404	Endpoint tidak ditemukan



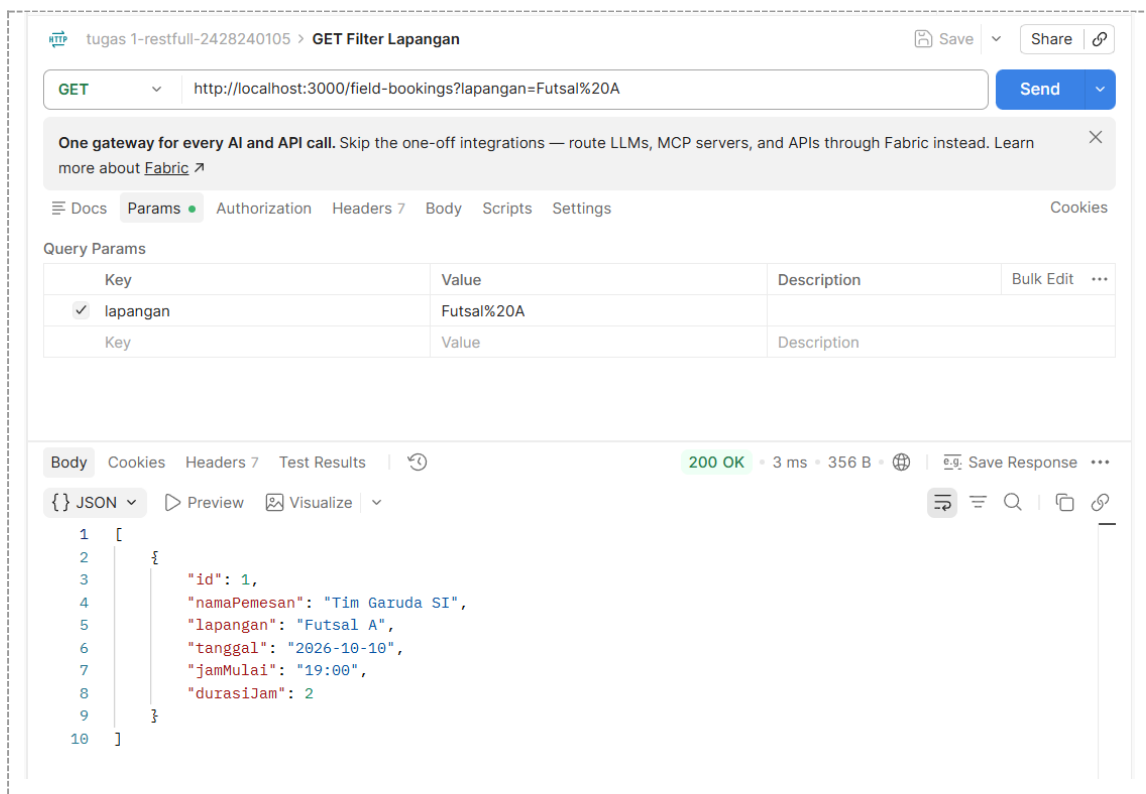
Gambar 11. Menampilkan sseluruh data booking



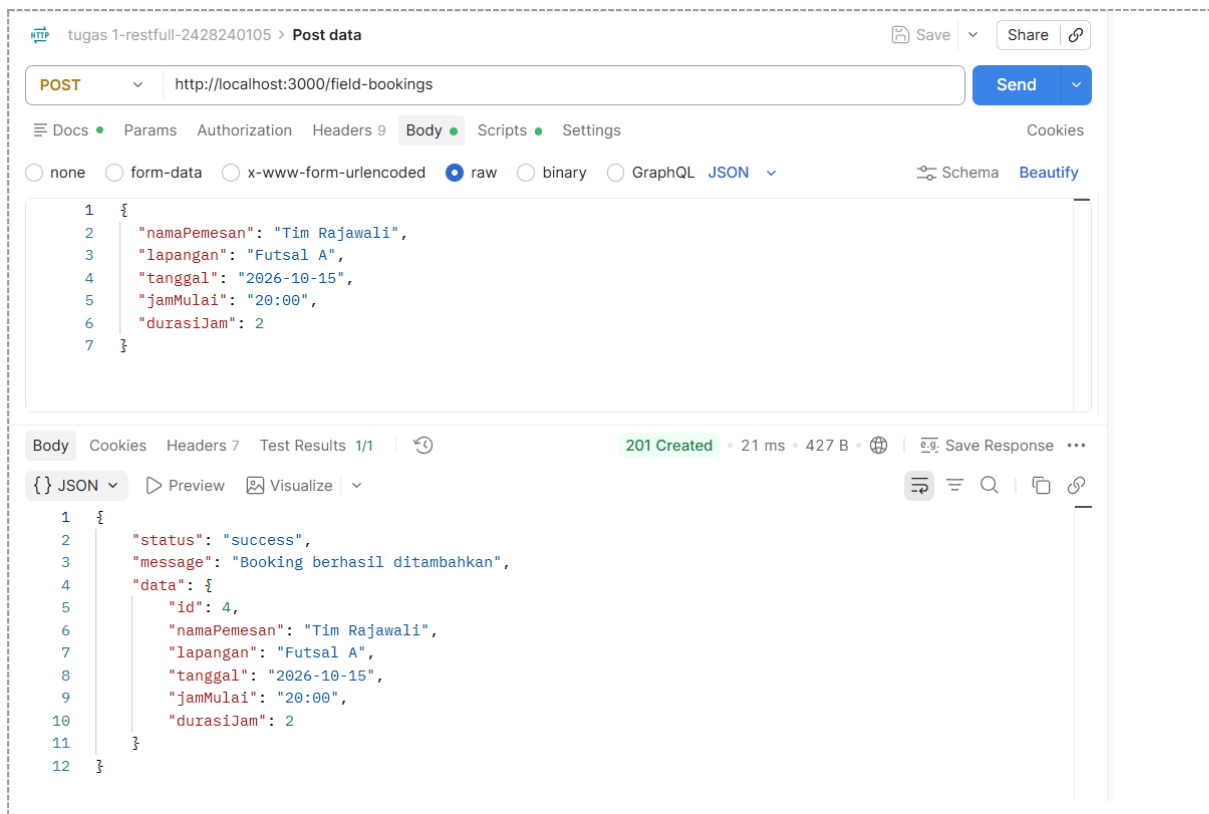
Gambar 12. GET berdasarkan ID



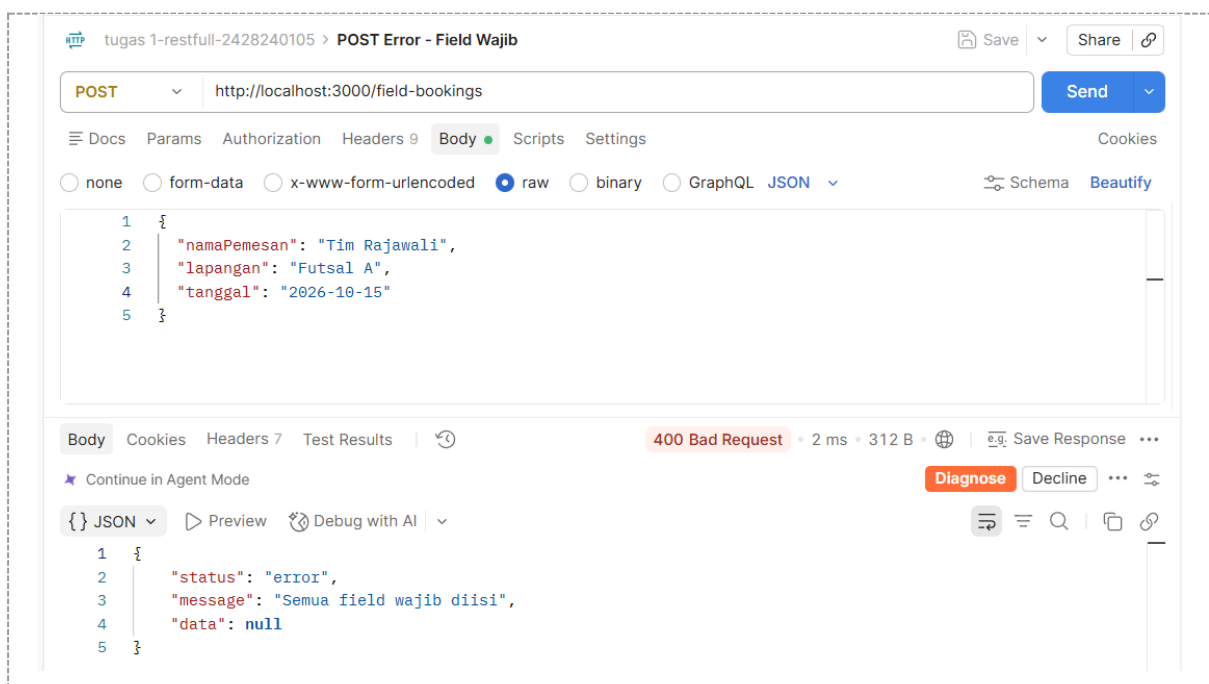
Gambar 13. GET /99 (404)



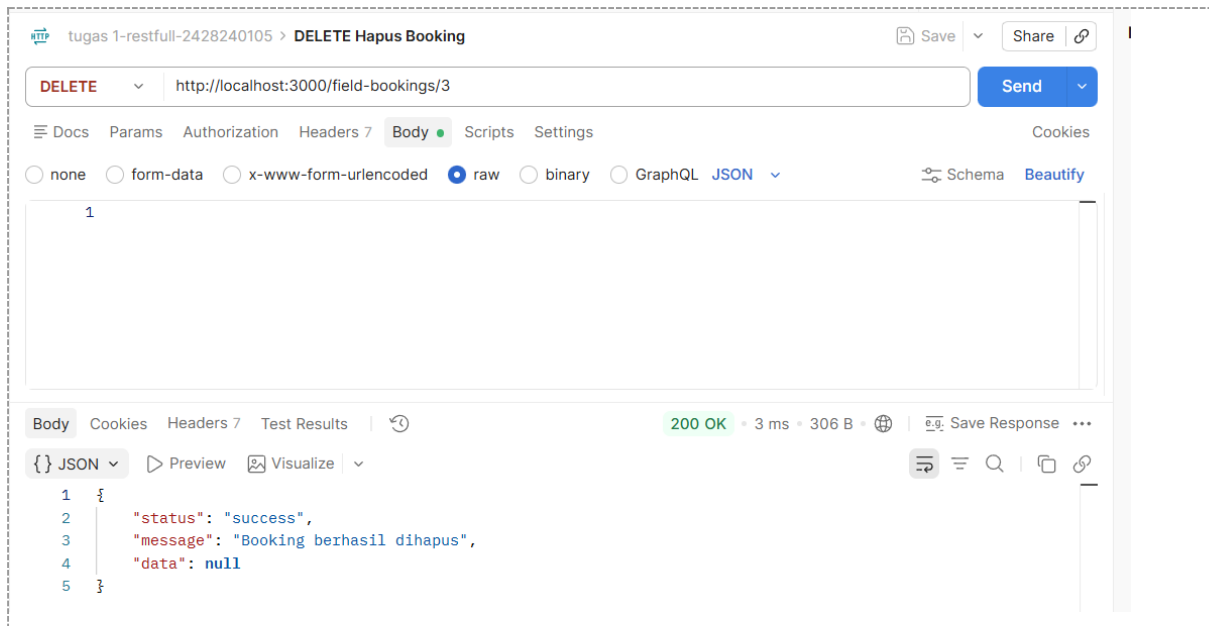
Gambar 14. GET Filter Lapangan



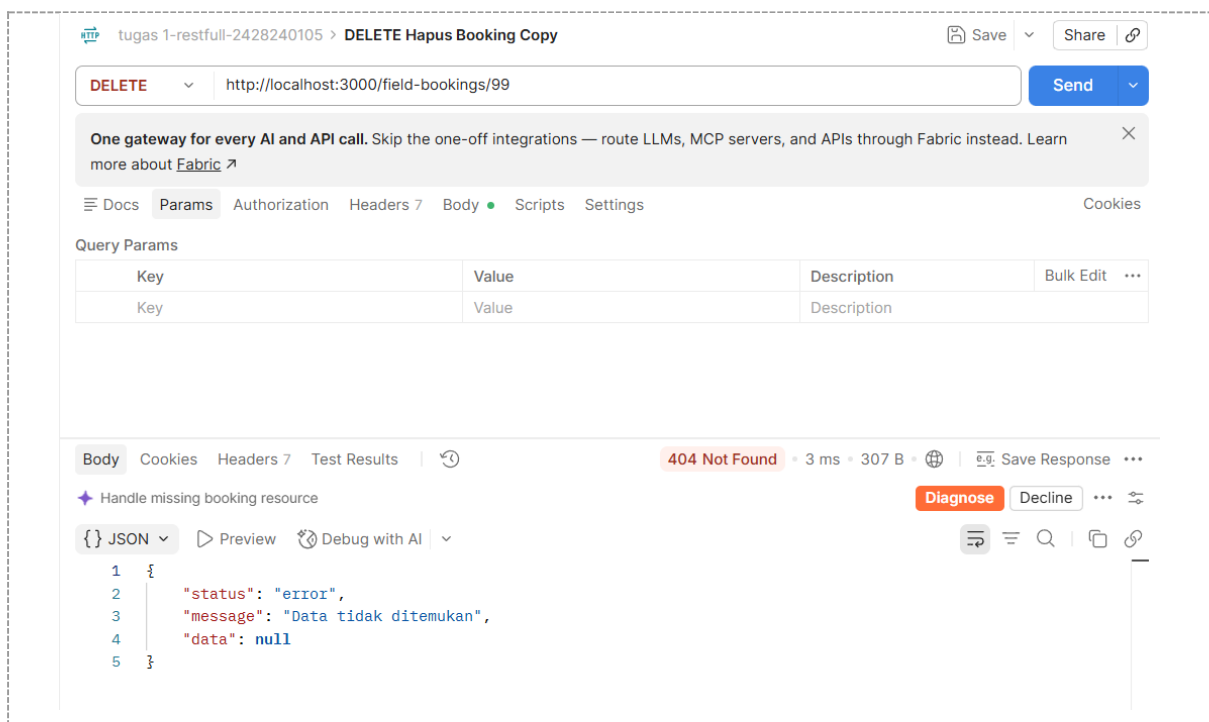
Gambar 15. POST /Tambah Data



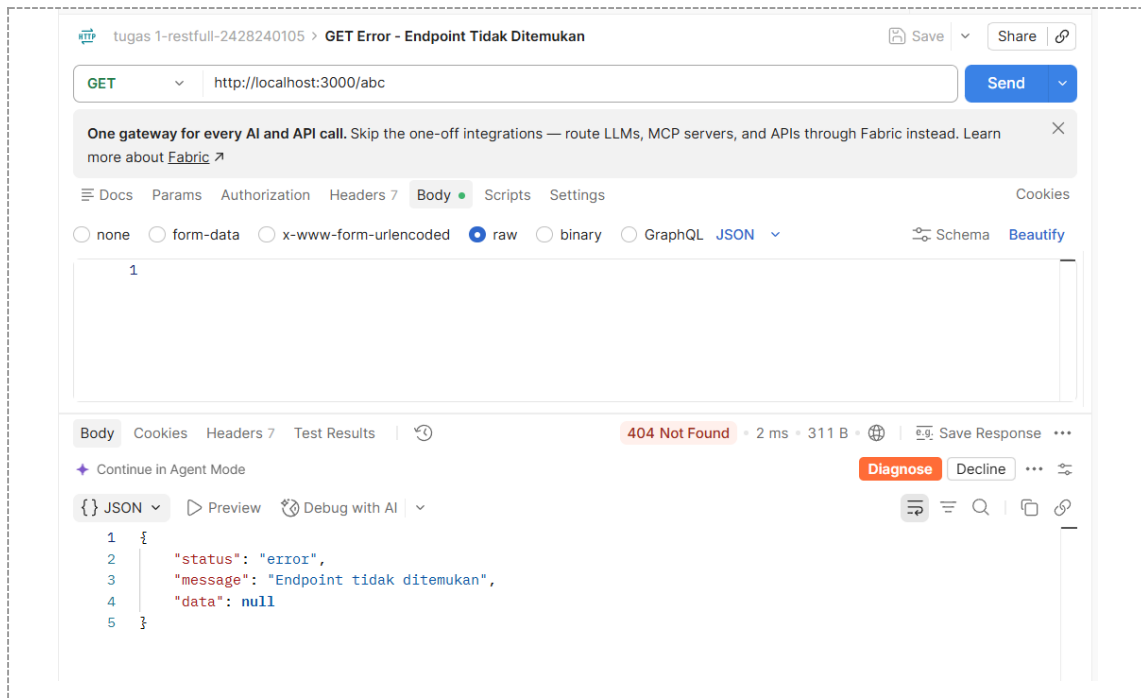
Gambar 17. POST /field-bookings dengan field wajib kosong (400)



Gambar 18. DELETE / Menghapus data ID 3



Gambar 19. DELETE / ID tidak ditemukan



Gambar 20. abc (catch-all)/endpoint tidak di temukan

VI. Pembahasan

A. Alasan Pemilihan Method HTTP dan Status Code\

Pada project RESTful API booking lapangan, method HTTP dipilih berdasarkan fungsi dari masing-masing endpoint. Method GET digunakan untuk mengambil data booking, baik seluruh data, data berdasarkan ID, maupun data berdasarkan filter lapangan. Method POST digunakan untuk menambahkan data booking baru. Method PUT digunakan untuk memperbarui data booking secara lengkap berdasarkan ID, sedangkan method DELETE digunakan untuk menghapus data booking berdasarkan ID.

Status code digunakan untuk menunjukkan hasil dari setiap request. Status 200 digunakan ketika request berhasil diproses, seperti saat mengambil, memperbarui, atau menghapus data. Status 201 digunakan ketika data booking berhasil ditambahkan. Status 400 digunakan ketika data yang dikirim tidak memenuhi validasi, misalnya terdapat field wajib yang tidak diisi. Status 404 digunakan ketika data atau endpoint yang diminta tidak ditemukan.

C. Kendala dan Solusi

Kendala	Solusi
Selama proses pembuatan RESTful API, terdapat beberapa kendala, terutama pada proses validasi data dan deployment. Pada validasi, perlu memastikan bahwa semua field wajib diisi dan durasiJam harus berupa angka. Selain itu, ID booking harus dibuat secara otomatis agar tidak terjadi ID yang sama. Kendala lainnya terjadi saat melakukan deployment ke Vercel karena aplikasi yang awalnya dijalankan secara lokal menggunakan port 3000 perlu disesuaikan agar dapat berjalan pada environment Vercel. Pengaturan aplikasi kemudian disesuaikan dengan menggunakan <code>process.env.PORT</code> dan <code>module.exports = app</code> sehingga aplikasi dapat dijalankan pada Vercel.	Kendala tersebut diatasi dengan melakukan pengecekan kembali terhadap kode dan menguji setiap endpoint menggunakan Postman. Validasi ditambahkan pada method POST dan PUT untuk memastikan data yang masuk sesuai dengan ketentuan. Untuk deployment, konfigurasi aplikasi disesuaikan dengan kebutuhan Vercel, kemudian project di-push kembali ke GitHub dan dilakukan redeploy. Setelah deployment berhasil, setiap endpoint diuji kembali menggunakan URL Vercel untuk memastikan API dapat digunakan dengan baik.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa RESTful API untuk sistem Booking Lapangan berhasil dibuat menggunakan Node.js dan Express.js. API telah menerapkan method GET, POST, PUT, dan DELETE sesuai dengan fungsi masing-masing endpoint.

API juga telah dilengkapi dengan validasi data, pembuatan ID secara otomatis, penggunaan parameter ID dan query filter, serta penanganan error menggunakan status code yang sesuai. Seluruh response API dibuat dalam format JSON sesuai dengan ketentuan yang diberikan.

Pengujian dilakukan menggunakan Postman pada endpoint yang telah di-deploy ke Vercel. Hasil pengujian menunjukkan bahwa endpoint dapat menjalankan proses

pengambilan, penambahan, perubahan, dan penghapusan data serta dapat menangani kondisi data atau endpoint yang tidak ditemukan.

Dengan demikian, praktikum ini memberikan pemahaman mengenai proses pembuatan, pengujian, pengelolaan source code menggunakan GitHub, dan deployment RESTful API menggunakan Express.js ke Vercel.

penambahan, perubahan, dan penghapusan data serta dapat menangani kondisi data atau endpoint yang tidak ditemukan.

Dengan demikian, praktikum ini memberikan pemahaman mengenai proses pembuatan, pengujian, pengelolaan source code menggunakan GitHub, dan deployment RESTful API menggunakan Express.js ke Vercel.

Daftar Pustaka

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

MDN Web Docs. (2026). HTTP request methods. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

MDN Web Docs. (2026). HTTP response status codes. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

MDN Web Docs. (2026). JSON. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Glossary/JSON>

Express.js. (2026). Express - Node.js web application framework. <https://expressjs.com/>

Node.js. (2026). Node.js Documentation. <https://nodejs.org/docs/latest/api/>

Lampiran

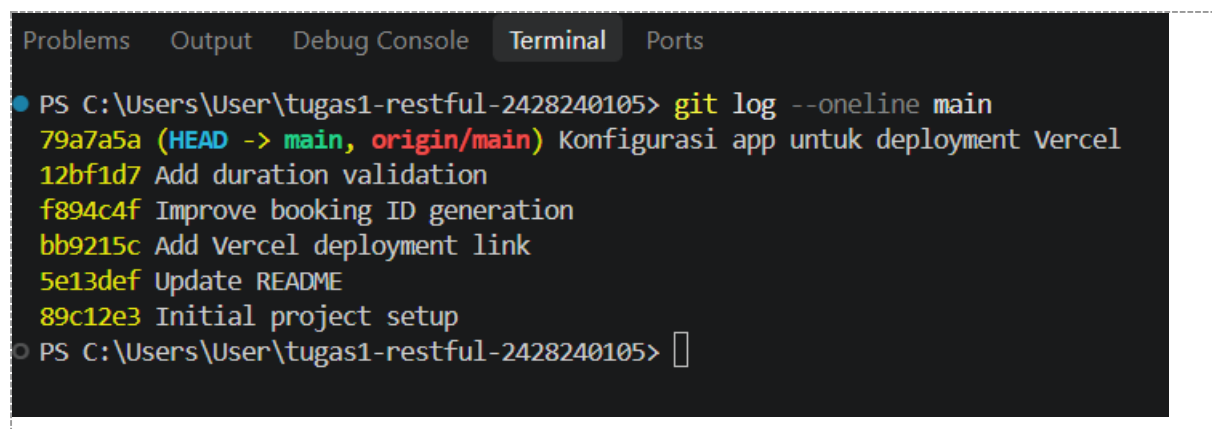
Link repository GitHub:

<https://github.com/revalina2428240105/tugas1-restful-2428240105>

Link deploy Vercel:

<https://tugas1-restful-2428240105.vercel.app/>

Riwayat commit (git log --oneline):



```
PS C:\Users\User\tugas1-restful-2428240105> git log --oneline main
79a7a5a (HEAD -> main, origin/main) Konfigurasi app untuk deployment Vercel
12bf1d7 Add duration validation
f894c4f Improve booking ID generation
bb9215c Add Vercel deployment link
5e13def Update README
89c12e3 Initial project setup
PS C:\Users\User\tugas1-restful-2428240105> 
```

Gambar 23. Hasil perintah git log --oneline